



Bachelor's Project in Nanoscience

August Baggesen Hilger

Deep Learning for PSF Localization

Region-based Deep Learning for Detection and Sub-Pixel Localization of Point Spread Functions in Heterogenous Backgrounds

June 10, 2025

Supervisor: Nikos Hatzakis

Region-based Deep Learning for Detection and Sub-Pixel Localization of Point Spread Functions in Heterogenous Backgrounds

August Baggesen Hilger

Single-molecule studies rely on efficient fluorophore localization and robust data analysis pipelines to provide unique insight into biochemical mechanisms and kinetics at a molecular level. Conventional methods for point spread function (PSF) localization require human calibration and are sensitive to changing environments, while current state-of-the-art Deep Learning solutions are restricted to comparably high signal-to-noise ratio (SNR) use cases in Single Molecule Localization Microscopy. Here, Faster R-CNN, a Region-based Deep Convolutional Neural Network for object detection, was successfully applied for accurate single-frame PSF detection and localization in heterogeneous noise environments with low to high SNR. Furthermore, an additional customizable regression head was implemented for the prospect of task-specific predictions. The model was trained on simulations of experimental data and achieves high detection and localization accuracy, outperforming a conventional Laplacian of Gaussian-method on simulated data. The model demonstrates promising performance on experimental data, indicating its potential although continued refinement and optimization will further enhance its capabilities.

1 Introduction

Single-molecule studies using fluorescence microscopy can shed light on biochemical processes and quantify the behavior of individual proteins or particles in ways that are otherwise inaccessible by conventional ensemble measurements.^{1,2,3,4} These studies are heavily dependent on robust image and data analysis methods,^{5,6} in particular the efficient localization of fluorophores.⁷ Emitters function as point sources of light due to their small size compared to the wavelengths of light used in fluorescence imaging, which causes emitters to show up on the microscope as point spread functions (PSFs). The optical effects that govern this phenomenon are understood well enough to allow researchers to take advantage of the shape of the PSF to perform accurate localization of emitters below the diffraction limit.⁸ Accurate localizations provide the fundamental base for downstream single-particle tracking (SPT)⁹ or super resolution image reconstruction through single-molecule localization microscopy (SMLM).⁵ Regardless of task, error prone detections of emitters will cascade faulty results through any given analytical pipeline, emphasizing the need for precise localization.

Conventional methods for PSF localization, such as the Laplacian of Gaussian-filter,¹⁰ Least-

squares Gaussian Fitting and Maximum Likelihood Estimation,⁷ are limited in that they require fine-tuning of parameters and lack flexibility across use cases. Machine Learning (ML) methods offer significant advantages in this respect, since ML algorithms are able to learn and adapt to data over time and recognize patterns that might otherwise be hidden. Deep Learning (DL) has become a popular tool for data analysis in fluorescence imaging,¹¹ owing to its capabilities in terms of large-scale processing and analysis of complex, multidimensional data, and since it is often possible to realistically simulate ground truth data that can be used for training DL models. Convolutional Neural Networks (CNNs) stand out in recognizing and extracting features from images, and they have been used by several studies in recent years, including DeepStorm3D¹² and DECODE,¹³ to extract emitter coordinates for SMLM. These solutions however lack applicability outside of the domain of SMLM.

The aim of this project is to implement and train a deep learning model for efficient PSF localization, using an architecture designed with the prospect of addressing a broader range of challenges, such as segmentation or multiclass object classification. To achieve this, a script for simulating ground truth images featuring varying signal-to-noise ratios, emitter distributions and hetero-

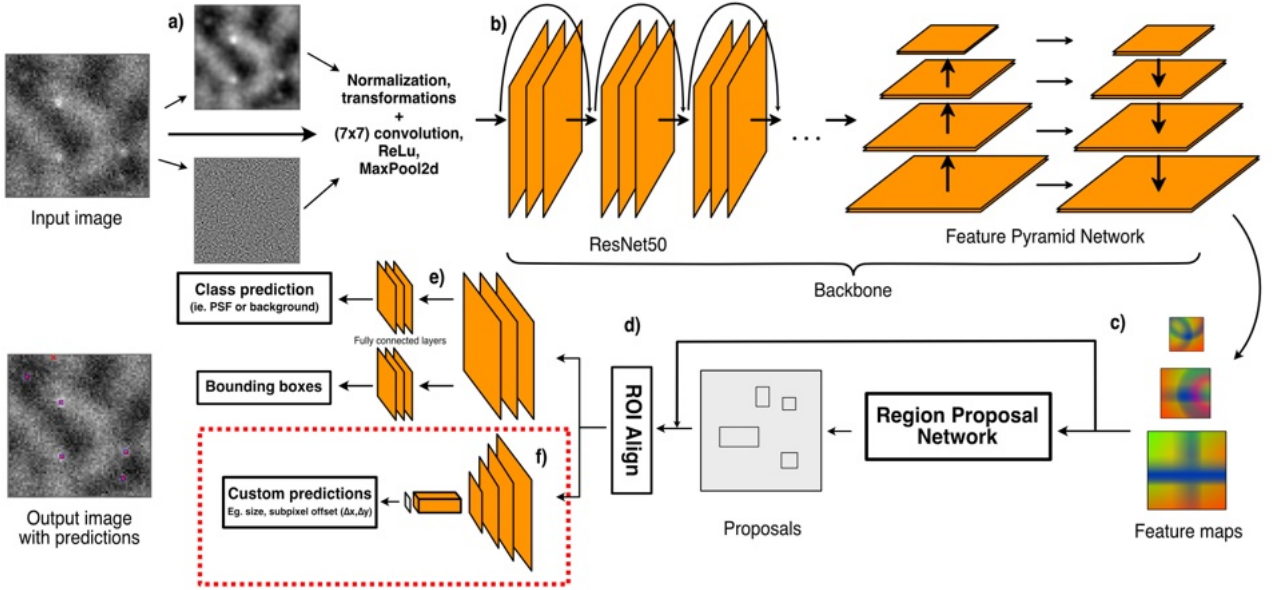


Figure 1: Model architecture. **a)** Images are standardized and undergo initial transformations as in the standard PyTorch Faster R-CNN implementation. **b)** The backbone consists of a ResNet50 network, which is a 50-layer CNN with bottleneck residual blocks, and a Feature Pyramid Network that outputs feature maps in different sizes. **c)** Region Proposal Network generates Region-of-Interest (RoI) proposals from the feature maps. **d)** MultiScaleROIAlign extracts fixed size 7x7 feature maps for the proposed regions using bilinear interpolation to retain pixel-accurate information. This provides a consistent representation for each region. **e)** TwoMLPHead performs classification and bounding box regression using fully connected layers. Finally class scores and bound box coordinates are output. **f)** A custom head was implemented to perform sub-pixel localization, but it was found to be more efficient and straightforward to infer PSF centers from box coordinates. Still, this head could easily be designed to predict other PSF or non-PSF class-specific properties or behavior in the future.

geneous noise backgrounds was built from scratch and used to train the Faster R-CNN model for PSF localization, utilizing the PyTorch package. Since data simulation is fast and random, images and ground truths can be generated on demand during training, ensuring efficient training and a diverse training set, without the need for data augmentations and transformations. The choice to base this work on a Faster R-CNN architecture was made for several reasons. First, deep CNNs are already proved to be effective in sub-pixel localization.^{11,14} Second, the architecture is in theory already geared to be able to include non-PSF detection and classification, ideally promising a network that is able to detect and recognize PSFs, cells, organelles and more all at once. Third, it is easy to add branches to the algorithm pipeline with minimal overhead, in the same manner as binary mask segmentation in Mask R-CNN.¹⁵ This could enable built-in fingerprinting of non-PSF objects, including predictions for object size, sub-pixel offsets within the bounding box and of course segmentation.¹⁶

2 Results and Discussion

For an in-depth discussion about architecture choices and details on simulation, training and evaluation please see the Methods section.

2.1 Network architecture

The architecture of the network is shown in Figure 1 and is based on Faster R-CNN,¹⁷ which is designed for general object instance detection and classification. On top of the Faster R-CNN framework, initially, was added a custom head, similar to the way a head is added in Mask R-CNN,¹⁸ consisting of convolutions, fully connected layers, and Smooth L1 Loss to predict the sub-pixel location of PSFs. However, it was found to be more efficient to simply infer the PSF centers directly from the float-value bounding box coordinates predicted by the Faster R-CNN framework. This was achieved by generating the ground truth bounding boxes as squares placed around the PSF centers. Inside the Faster R-CNN head, bounding box regression happens with a Smooth L1 Loss function,

$$\mathcal{L}(x, y) = \begin{cases} 0.5(x - y)^2 / \beta, & \text{if } |x - y| < \beta \\ |x - y| - 0.5\beta, & \text{otherwise} \end{cases} \quad (1)$$

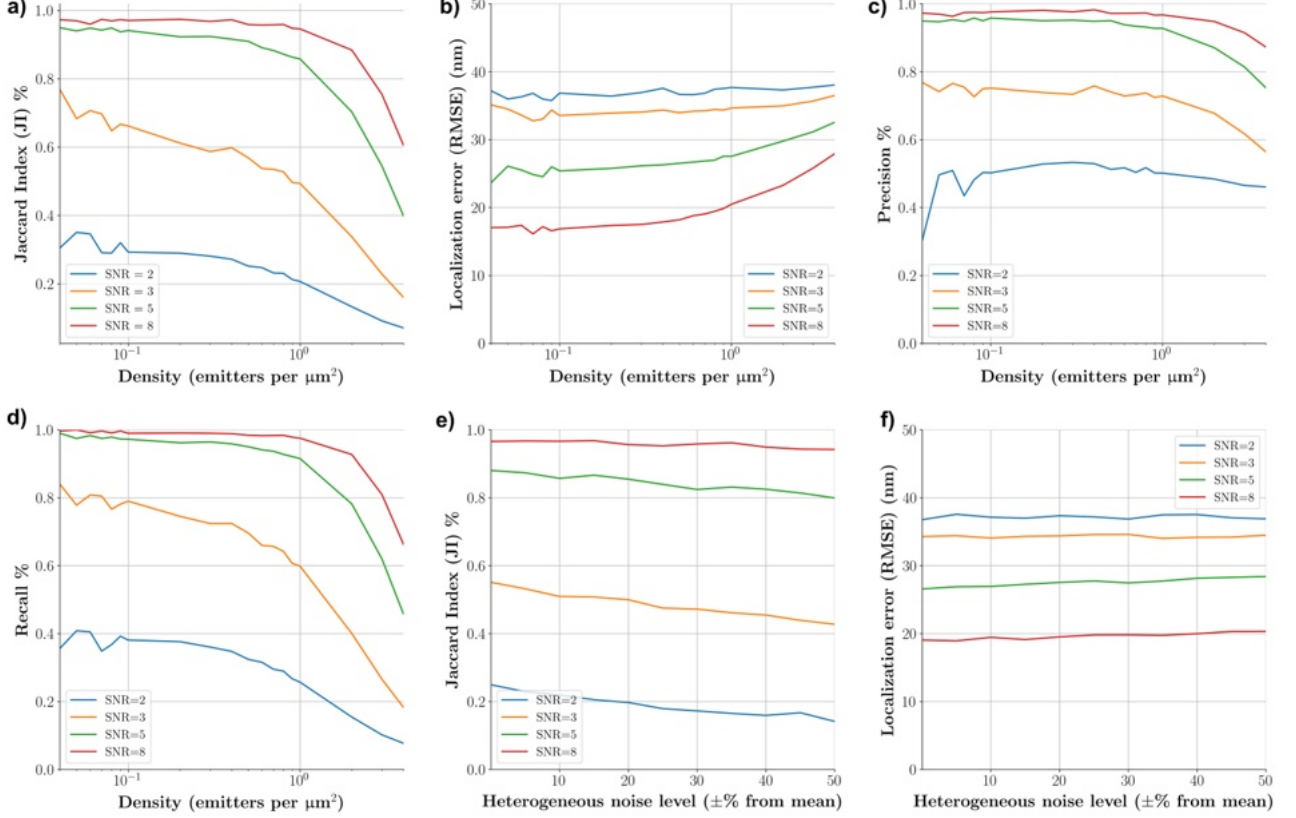


Figure 2: Performance metrics. True predictions count when IoU > 0.5 and assigned score > 0.7. Predictions were made on 300 images per SNR. **a-d)** Performance in homogeneous noise environments. The model has high detection accuracy at medium and high SNRs and at low and medium densities. Recall is higher than precision, which might indicate that a higher score threshold should be used to avoid false predictions. Localization error is low, definitely below the diffraction limit. **e-f)** Performance in heterogeneous noise environments. As expected, performance declines, but relatively good accuracy is retained. Metrics collected from images with fixed density of 0.5 emitters per μm^2 .

where x is the predicted bounding box coordinate, y is the ground truth coordinate and $\beta = 1/9$. This loss function is brought forward, since it is highly important for the model’s ability to accurately predict PSF centers with sub-pixel accuracy. It works because when errors are relatively large ($|x - y| > \beta$), the model is not penalized too heavily, and in the other case the squared term ensures smooth optimization.

Model accuracy was found to increase when using standardization for feature scaling, $x' = (x - \mu)/\sigma$, as opposed to min-max normalization $x' = (x - x_{min})/(x_{max} - x_{min})$, shown in Figure 3. Providing additional image preprocessing filters via two auxiliary channels in the three channel-input also resulted in higher accuracy. Specifically, a Gaussian filter was applied in the second channel and a Laplacian filter in the third channel. Further, activating five trainable layers of the ResNet50 backbone, rather than three, was found to give better performance (not shown here).

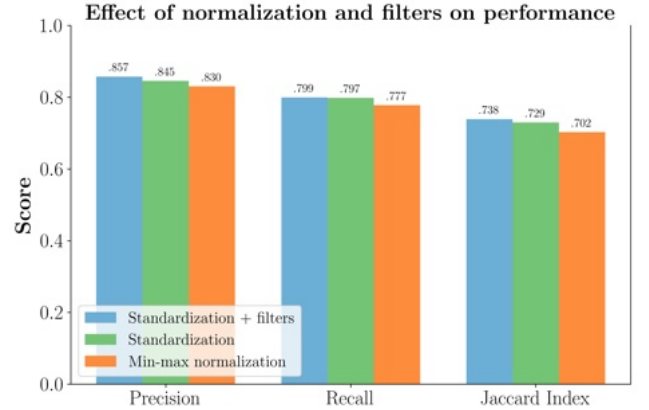


Figure 3: Performance depends on preprocessing. Comparison of how different preprocessing actions affect performance. The best performance is achieved when using mean standardization and supplying the backbone with additional Gaussian- and Laplacian-filtered channels. Metrics were gathered from 400 images with highly random densities and background levels and ideal SNR levels ranging from 2-10.

2.2 Model performance

The performance of the model was validated quantitatively on simulated data and qualitatively on experimental data. The two most important per-

formance metrics are accuracy of detection and localization accuracy. Accuracy of detection depends on the relationship between true positives (TP), false positives (FP) and false negatives (FN) and can be quantified by precision: $TP/(TP+FP)$ and recall: $TP/(TP+FN)$. The Jaccard Index (JI) is a metric that attempts to reflect the balance between precision and recall by incorporating both false positives and false negatives in the score: $JI = TP/(TP+FP+FN)$.

Figure 2 shows relevant performance metrics on simulated data for different densities and signal-to-noise ratios (SNRs). As expected, the model generally performs better at lower emitter densities and higher SNRs. Detection accuracy improves significantly going from low ($\approx 2-3$) to medium (≈ 5) SNR. All metrics were collected using a score threshold of 0.7, chosen by trial and error, as score thresholds around 0.66 – 0.70 were found to yield the highest detection accuracy (JI) overall. Ideally, fine-tuning of the score threshold would be required for each use case. For large SNRs at low and medium densities, a score threshold of 0.9 was found to give very high JI $\approx 0.98-0.99$, by eliminating almost all false positives. The average localization error of all true positives, measured in root mean square distance (RMSE), reaches 19 nanometers at low densities and high SNR, approaching the resolution of the best SMLM techniques, which is around 10-20nm.⁵

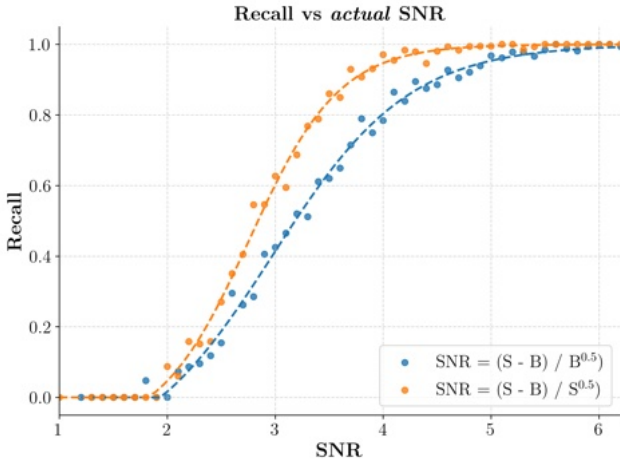


Figure 4: Recall vs actual SNR values. SNR is calculated from each individual emitter. Metrics collected on images with homogeneous backgrounds and only one emitter per image.

It is important to note that the SNR values represented in Figure 2 are the *ideal* SNR values used to simulate images. When Poisson noise is

applied to an image after PSF generation, pixel values change, so that the *actual* SNR values of the PSFs in the image are random, but still mainly distributed around the ideal SNR value. Eg. an image generated with ideal $SNR = 3$ will result in actual SNRs $\approx 1 - 5$. Figure 4 shows the model’s recall against actual SNR, extracted from images with homogeneous backgrounds that contain a single emitter. Note that two ways of calculating SNR are considered (see Methods for details). The model is moderately successful at detecting emitters at low-medium SNR ($\approx 3 - 4$) and performs very well at higher SNR.

The model was also tested with heterogeneous noise levels ranging from 0% to 50%, meaning that the brightest and dimmest part of the background had pixel values $x\%$ larger and smaller than the mean background value, respectively. Performance was good on heterogeneous background, with minor losses in detection and localization accuracy compared to homogeneous background, as illustrated in Figure 2 e-f). As expected, detection accuracy (JI) decreases with increasing noise level, but the model still shows good performance at high SNR.

Figure 8 and Figure 18 in appendix show some example images of model predictions on simulated images with homogeneous and heterogeneous noise.

Experimental data

To test performance on experimental data, the model was employed for detection and localization of PSFs in three real scenarios: 1) Lipid nanoparticles (LNPs) carrying antisense oligonucleotides (ASOs), courtesy of Theodor Redvall, 2) Transferrin proteins in HeLa cells and 3) Lysosomes in HeLa cells, both courtesy of Sara V. Bleshø.

Since there is no ground truth in the same sense as for simulated images, performance has to be assessed qualitatively. Overall, the model works well, though it seems to suffer from making false predictions (Figures 10 through 15 in appendix). Part of this issue could stem from the fact that the model was not exposed to images without emitters during training. However, during inference, large images are processed as a series of 64×64 patches, some of which may be empty. As a result, the model may hallucinate detections when presented with truly empty patches. Also, for now at least, the model is not suited to deal

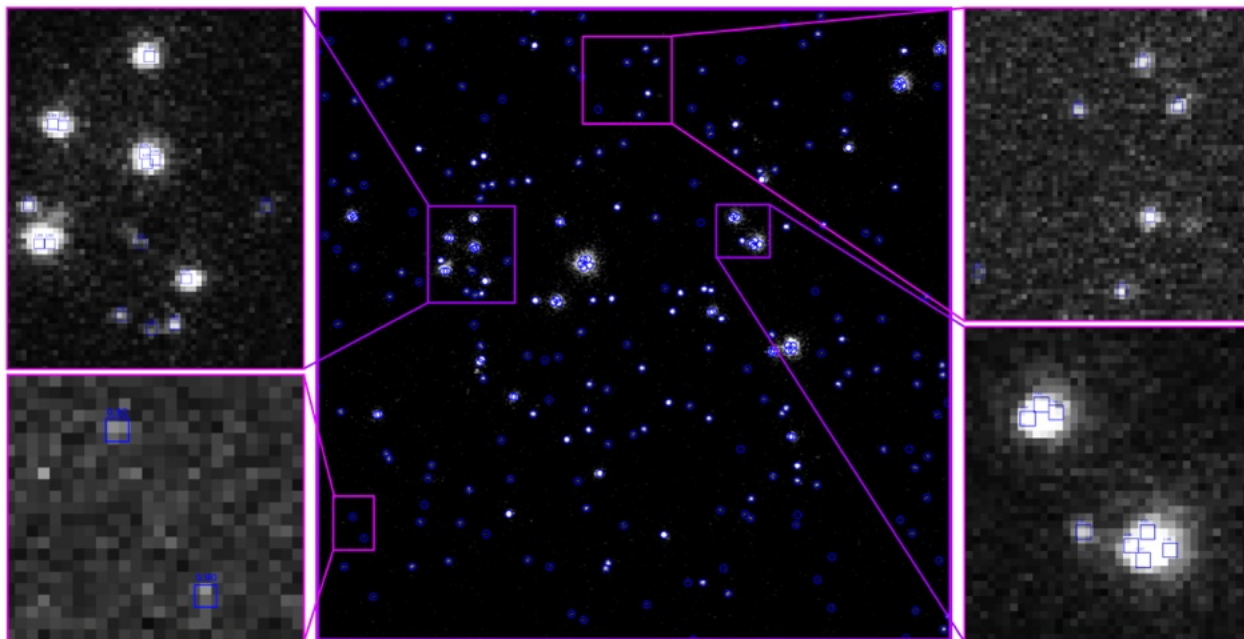


Figure 5: Localization of fluorescently labeled LNPs. The figure shows predicted emitter positions on a very high-SNR TIRF image of LNPs. Select regions of interest are shown in insets with increased brightness and contrast for clarity. Overall, the model is deemed to perform well, though there could seem to be some false positives (eg. in top right inset). For very large, or dense, emitters, the model predicts several bounding boxes within the "same" spot (top left and bottom right inset), since the model is trained to know only one size of PSF. The number above each bounding box prediction represents the score that the model has assigned to that prediction. The predicted emitter on the bottom left has a peak of 2650 and thus a theoretical SNR around 11-12. The background mean is around 2100 and the raw maximum pixel value is 64000. Before model prediction, the image was clipped to [0,8000] and standardized. Image courtesy of Theodor Redvall.

with larger spots, e.g. spots stemming from aggregations of LNPs or molecules. This is illustrated in Figure 5, where it makes several predictions inside large spots. This is not surprising though, as the model has not been trained to predict bounding boxes with any sizes other than 2x2. The model is able to work in both very high (Fig. 5) and medium-low (Figs. 10 through 15) SNR conditions.

Comparison to Laplacian of Gaussian

The Laplacian of Gaussian (LoG) is an image filter commonly used for edge detection and feature extraction.¹⁰ The idea is to reduce image noise using Gaussian blur, and then to use the Laplacian operator, i.e. taking the second derivative, to identify pixels or areas with steep changes in intensity, which indicate a blob or an edge. Gpu-tracking¹⁹ is an in-house Python package, which implements this algorithm for PSF localization, making it a meaningful benchmark method to compare against. The model and gpu-tracking were compared both quantitatively with simulated data and qualitatively with experimental data. On the simulated data, the model significantly

outperforms gpu-tracking (Fig. 7). This is not surprising in itself - the model is on home turf - but the inefficiency of the LoG algorithm is unexpected. Ideally, the two methods should be tested on third party simulated data, such as data from the SMLM Challenge²⁰, but unfortunately this was not possible due to time constraints. Upon investigation, it seems that the LoG method is very challenged by heterogeneous noise environments (see Fig. 18 in appendix).

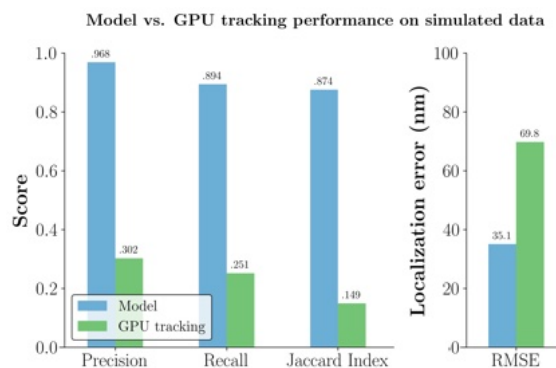


Figure 7: Performance comparison of the model and gpu-tracking on simulated data. Gpu-tracking settings were set through trial and error to optimize the average JI score. Predictions are counted as TP if they are at less than 1 pixel length away from a true emitter position.

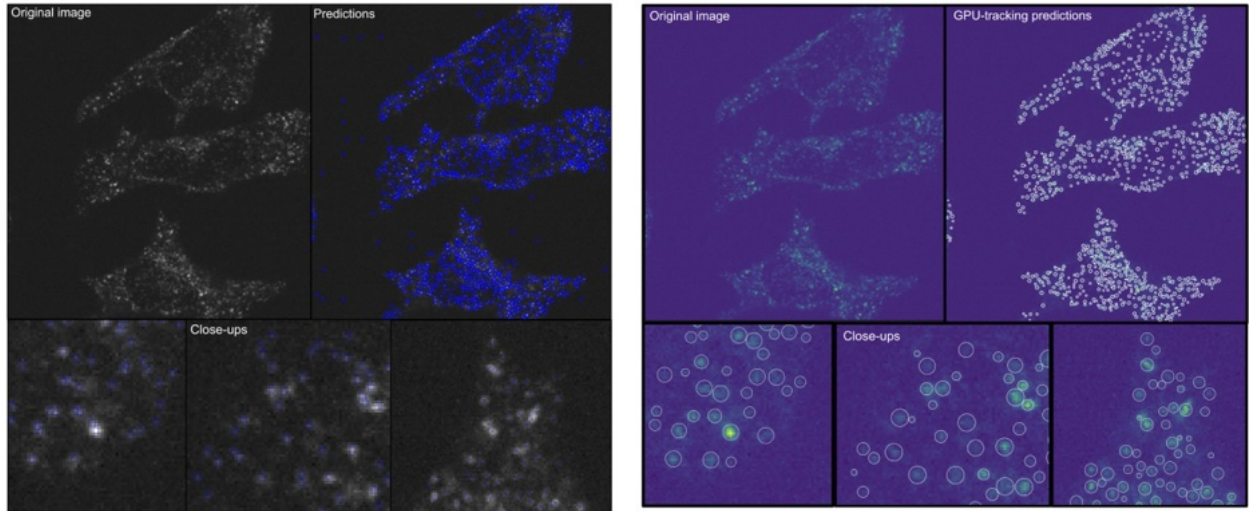


Figure 6: Side-by-side comparison of model and gpu-tracking localization of transferrin in HeLa cells. Left: Model predictions. The model makes many, what looks to be, false positive predictions outside of the cells. This begs the question of whether these predictions are actually false positives, or if they correspond to PSFs that are simply invisible to the naked eye due to very low SNR. After brightness and contrast enhancement (appendix Fig. 17) and careful visual examination, the "spots" are still hidden, suggesting they might indeed be false positives. Right: Gpu-tracking predictions. Gpu-tracking seems to perform better both in terms of avoiding false positives and false negatives. Image courtesy of Sara V. Bleshøy

On experimental data however, the two approaches are equally good, or gpu-tracking is better (Fig. 6), avoiding what looks like false positives to a larger degree. This is a purely qualitative assessment of course, but it does show that there is still room for improvement in terms of consistency. In the immediate future, it would be interesting to test the methods on experimental data with highly heterogeneous backgrounds, and also to actually expose the model to patches without emitters during training, to see if that fixes the problem with false positives.

3 Future directions

This project successfully implemented PSF localization, but there remains room for improvement of accuracy of detection, particularly on experimental dataset. Further, to extend the systems capabilities to include detection and classification of non-PSF objects, eg. cells, microtubules or membranes, there is a need for more complicated and intricate training data. This could either involve advanced simulated data approximating non-PSF objects, or experimental datasets with ground truths labeled by humans. Future work should also address 3D data and add a temporal dimension to allow single particle tracking and identification of eg. splitting and merging events. For

an efficient implementation, the model might be restructured to be able to jointly perform spatio-temporal localization,²¹ allowing to take full advantage of the temporal context, instead of simply stitching together a frame-by-frame analysis.

4 Methods

4.1 Architecture considerations

Choosing an architecture for a deep learning network is crucial for performance and should be tailored according to the task at hand. One has to consider several features - the kind of modules and layers to use, their order and depth of the network.^{22,23} A significant amount of work has gone into understanding current state-of-the-art solutions for PSF localization, particularly in SMLM, and into exploring ML approaches for computer vision and object detection in general.²⁴ Here is a short review of relevant studies, followed by a description of how they influenced the architectural choices for the model in this work.

In the context of SMLM, many groups are working on using DL for PSF localization for the purpose of constructing super-resolution images. In 2018, Nehme et al. published DeepStorm,²⁵ using a deep CNN to perform localization and extract super-resolution images. This was followed

up by the 3D-based DeepStorm3D in 2020.¹² In 2021, Speiser et al. released DECODE,¹³ which uniquely incorporates predictions of probability of detection and uncertainty of localization into their loss function, leading to even better results.

Faster R-CNN (2015)¹⁷ is a Region-based Convolutional Neural Network (RCNN) which combines the capability of a CNN to pick out relevant features in images with a Region Proposal Network (RPN), that is able to predict regions of interest (ROIs) in images to perform object instance detection and classification. Faster R-CNN was an improvement both in terms of speed and accuracy compared to R-CNN²⁶ and Fast R-CNN.²⁷ In 2017, Mask-RCNN¹⁸ extended Faster R-CNN and added a binary mask prediction branch to perform object instance segmentation. YOLO (You Only Look Once)²⁸ is another detection algorithm, which is known for its exceptional speed. YOLO also predicts bounding boxes and class labels, but it is different from the region-based algorithms, since YOLO consists of a CNN predicting bounding boxes and class labels in a single pass through the network. YOLO is faster than Faster-RCNN, but some report higher accuracy in Faster R-CNN,²⁹ which should be prioritized in PSF localization.

In the end, the choice was made to base this work on the Faster R-CNN model, even though the recent successful applications of deep learning for single molecule localization, DeepStorm3D and DECODE, do *not* employ Region-based CNNs. Instead, they are highly customized models that are optimized to tackle the particular challenges posed by the PSF localization problem. Though effective, these methods are also inherently limited to applications in SMLM, since their goal is solely accurate localization of blinking emitters. Here, however, the aim was to lay the groundwork for a system that could be able to perform multi-class object detection on fluorescence microscopy data.

Another interesting approach to localization is taken by DeepBlink,³⁰ working similarly to YOLO, where the input image is mapped into smaller regions in a grid, for which it predicts the probability and location of a spot.

4.2 Signal-to-noise ratio

Performance as a function of SNR is one of the primary metrics for evaluating the effectiveness of a PSF localization model. In fluorescence mi-

croscopy, SNR is calculated as the difference between the peak of the signal S and the background mean B , divided by a representative noise level σ .¹⁵

$$S/N = (S - B)/\sigma. \quad (2)$$

Since the standard deviation of a Poisson distribution is equal to the square root of the mean, SNR can be defined as

$$\text{SNR}_{\text{bg}} = \frac{S - B}{\sqrt{B}}, \quad (3)$$

which reflects the strength of the signal compared to the standard deviation of the background. Cheezum et al.,¹⁵ however, argue that this is not appropriate. Instead, they argue that the stronger noise level in the actual signal should be used, since it has a larger influence on the shape of the PSF and causes inaccuracy in localization. Sbalzarini et al.³¹ used what they call the most conservative estimate by simply dividing by the maximum intensity of the peak, in theory accounting for shot-noise coming from both the background and the signal:

$$\text{SNR}_{\text{sig}} = \frac{S - B}{\sqrt{S}}. \quad (4)$$

Unless otherwise specified, the definition in (3) has been used in this work.

4.3 Validating the model

To calculate the number of emitters for a given density, a pixel size of 100nm is assumed, for ease-of-use and since pixel sizes in fluorescence microscopy are often around 65-180nm.^{32,33} Also, many simulated datasets from the SMLM challenge²⁰ use a pixel size of 100nm. This means, for example, that a 64x64 image with a density of $1\mu\text{m}^2$ corresponds to 40 emitters and a density of $0.05\mu\text{m}^2$ corresponds to two emitters.

Localization error in an image is often quantified by the root mean square error (RMSE):¹¹

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_i^N (\hat{x}_i - x_i)^2}, \quad (5)$$

where N is the number of true positives, $\hat{x}_i$ are the predicted positions and x_i are the ground truth positions.

4.4 Simulating ground truth data

The physics behind the formation of PSFs is complex but well enough understood to allow simulation of realistic approximations.³⁴ To simulate ground truth data for training the model, images are generated by drawing PSFs one by one on a background image containing Perlin noise, after which Poisson noise is applied. Image formation depends on several parameters: Background mean, Perlin noise intensity, number of emitters, desired SNR and the PSF width.

Approximating the point spread function

Choosing a mathematical model for approximating the PSF is a trade-off between accuracy on the one hand, and simplicity and cost of computation on the other.⁸ The exact shape of a PSF will depend both on properties of the emitter and the microscope used to observe it.³⁴ Some highly realistic approximations take into account numerical aperture and interactions between the lens, coverslip and immersion oil.^{35,36} However, these formulas are tedious and complicated, which is why many choose to approximate the PSF with an Airy function or most often just with a simple Gaussian function, which captures the shape of the PSF sufficiently well, especially near the center.⁸ For this work, the PSFs were simulated using a Gaussian approximation:

$$I(x, y) = I_0 \exp\left(-\frac{(x - x_0)^2 + (y - y_0)^2}{2\sigma^2}\right), \quad (6)$$

where (x_0, y_0) is the fluorophore position, σ is the standard deviation of the PSF in pixels, and I_0 is the maximum intensity of the PSF.

This way, the script is able to generate PSFs with approximate signal-to-noise ratios (SNR). As discussed, noise can be defined as the standard deviation of the background intensity. The noise in the image is Poisson distributed (see next paragraph), which implies that the expected value is equal to the variance. Thus, $\text{SNR} = S/\sqrt{\lambda}$, where λ is the mean of the background, and S is the peak of the signal above the background, $S_{\max} - \lambda$. In practice, the mean of the background and the *ideal* SNR are used to calculate the intensity of the PSF, which is generated accordingly. However, after adding Poisson noise to the idealized image, the peak of the PSF will likely have changed. Hence, the *actual* SNR values are calculated after image generation using the actual highest peak intensity

near the PSF center, and the background mean sampled from a donut-shape around the PSF.

Simulating signal and noise

Accurate simulation of noise is important for achieving realistic images. Noise can stem from both the physical imaging process and from limitations on the detector. Some relevant noise sources in fluorescence microscopy are read noise, thermal accumulation of electrons called dark current, and gain noise in EMCCD cameras.³⁷ But the most important kind of noise to consider is shot noise.⁸ The signal is in essence a random variable with a Poisson distribution - all photons arrive at the camera with the same probability. This is called shot-noise, and it is unavoidable due to the nature of light, which is why shot-noise is considered the most important contributor of noise in this system, since it in practice defines a lower limit to the signal-to-noise ratio.³⁸ Since the Poisson distribution is additive, shot-noise is realistically simulated by adding Poisson noise to an image containing a Perlin noise-background and the ideal PSFs.

To accurately model the noise and intensity of the signal, one should first apply shot-noise to the ideal number of photons coming from the emitter, then multiply by the quantum efficiency (QE) - the ratio of photons that are converted into photoelectrons in the detector - and apply other noise sources. In this simulation, only shot noise is considered and the QE is assumed to be 1. The reasoning is that since the model will be trained on a very wide range of pixel values anyway, it is adequate to simulate data in arbitrary units. Speiser et al.¹³ report that they did indeed find this reasoning to be true in their own similar simulation.

4.5 Training of the model

The model was trained on an Nvidia RTX A5000 24GB RAM for 7 hours. Training was done using a learning rate of 1e-4, the AdamW optimizer and batch size 8.

The simulated data for training was generated using the following parameters: Min/max no. of spots = [2;80], mean $\sigma = 1.5$, std. of $\sigma = 0.1$, min/max ideal SNR = [2;20], min/max base noise = [200, 10000]. The model was trained on 64x64 images. A script for image patching was incorporated, so the model is able to take images of arbitrary size as input and still perform predictions.

5 Acknowledgments

I would to thank my immediate supervisor, Steen Bender, for his indispensable guidance and hands-on help with coding and bug fixes, without which the project would be far from where it is now. Thank you Georgios Sevastakis, Marcus W. Dreisler and António R. Maschio for insightful dis-

cussions and helping with bug fixes. Thank you Theodor Redvall and Sara V. Bleshøy for providing experimental data for validating the model. Finally, I want to thank my supervisor, Professor Nikos Hatzakis, for the opportunity and for insightful discussions and directions related to the project.

References

- [1] MJ Saxton and K Jacobson. “Single-particle tracking: applications to membrane dynamics”. In: *Annual review of biophysics and biomolecular structure* 26 (1997), pp. 373–399.
- [2] Alessandro Borgia, Philip M Williams, and Jane Clarke. “Single-molecule studies of protein folding.” In: *Annual Review of Biochemistry* 77 (2008), pp. 101–125. DOI: 10.1146/annurev.biochem.77.060706.093102. URL: <http://dx.doi.org/10.1146/annurev.biochem.77.060706.093102> (visited on 06/08/2025).
- [3] Henrik D Pinholt et al. “Single-particle diffusional fingerprinting: A machine-learning framework for quantitative analysis of heterogeneous diffusion.” In: *Proceedings of the National Academy of Sciences of the United States of America* 118.31 (Aug. 2021). ISSN: 0027-8424. DOI: 10.1073/pnas.2104624118. URL: <http://www.pnas.org/lookup/doi/10.1073/pnas.2104624118> (visited on 06/08/2025).
- [4] Avelino Javier et al. “Persistent super-diffusive motion of Escherichia coli chromosomal loci.” In: *Nature Communications* 5 (May 2014), p. 3854. DOI: 10.1038/ncomms4854. URL: <http://dx.doi.org/10.1038/ncomms4854> (visited on 06/09/2025).
- [5] Ismail M Khater, Ivan Robert Nabi, and Ghassan Hamarneh. “A Review of Super-Resolution Single-Molecule Localization Microscopy Cluster Analysis and Quantification Methods.” In: *Patterns (New York, N.Y.)* 1.3 (June 2020), p. 100038. ISSN: 26663899. DOI: 10.1016/j.patter.2020.100038. URL: <https://linkinghub.elsevier.com/retrieve/pii/S266638992030043X> 7D (visited on 06/16/2020).
- [6] Jianli Liu et al. “Machine Learning for Single-Molecule Localization Microscopy: From Data Analysis to Quantification.” In: *Analytical Chemistry* 96.28 (July 2024), pp. 11103–11114. DOI: 10.1021/acs.analchem.3c05857. URL: <http://dx.doi.org/10.1021/acs.analchem.3c05857> (visited on 06/08/2025).
- [7] Hao Shen et al. “Single particle tracking: from theory to biophysical applications.” In: *Chemical Reviews* 117.11 (June 2017), pp. 7331–7376. DOI: 10.1021/acs.chemrev.6b00815. URL: <http://dx.doi.org/10.1021/acs.chemrev.6b00815> (visited on 03/18/2025).
- [8] Alex Small and Shane Stahlheber. “Fluorophore localization algorithms for super-resolution microscopy.” In: *Nature Methods* 11.3 (Mar. 2014), pp. 267–279. DOI: 10.1038/nmeth.2844. URL: <http://dx.doi.org/10.1038/nmeth.2844> (visited on 03/18/2025).
- [9] François Simon, Lucien E. Weiss, and Sven van Teeffelen. “A guide to single-particle tracking”. In: *Nature Reviews Methods Primers* 4.1 (Sept. 2024), p. 66. ISSN: 2662-8449. DOI: 10.1038/s43586-024-00341-3. URL: <https://www.nature.com/articles/s43586-024-00341-3> (visited on 06/08/2025).
- [10] Hui Kong, Hatice Cinar Akakin, and Sanjay E Sarma. “A generalized Laplacian of Gaussian filter for blob detection and its applications”. In: *IEEE transactions on cybernetics* 43.6 (2013), pp. 1719–1733.

- [11] Jian Mao and Hua He. “Deep learning in fluorescence imaging and analysis”. In: *Journal of Intelligent Medicine* 1.1 (Dec. 2024), pp. 42–62. ISSN: 2837-6749. DOI: 10.1002/jim4.17. URL: <https://onlinelibrary.wiley.com/doi/10.1002/jim4.17> (visited on 05/24/2025).
- [12] Elias Nehme et al. “DeepSTORM3D: dense 3D localization microscopy and PSF design by deep learning.” In: *Nature Methods* 17.7 (July 2020), pp. 734–740. ISSN: 1548-7091. DOI: 10.1038/s41592-020-0853-5. URL: <http://www.nature.com/articles/s41592-020-0853-5> (visited on 06/07/2025).
- [13] Artur Speiser et al. “Deep learning enables fast and dense single-molecule localization with high accuracy.” In: *Nature Methods* 18.9 (Sept. 2021), pp. 1082–1090. ISSN: 1548-7091. DOI: 10.1038/s41592-021-01236-x. URL: <https://www.nature.com/articles/s41592-021-01236-x> (visited on 04/04/2025).
- [14] Wei Ouyang et al. “Deep learning massively accelerates super-resolution localization microscopy.” In: *Nature Biotechnology* 36.5 (June 2018), pp. 460–468. DOI: 10.1038/nbt.4106. URL: <http://dx.doi.org/10.1038/nbt.4106> (visited on 03/18/2025).
- [15] M K Cheezum, W F Walker, and W H Guilford. “Quantitative comparison of algorithms for tracking single fluorescent particles.” In: *Biophysical Journal* 81.4 (Oct. 2001), pp. 2378–2388. DOI: 10.1016/S0006-3495(01)75884-5. URL: [http://dx.doi.org/10.1016/S0006-3495\(01\)75884-5](http://dx.doi.org/10.1016/S0006-3495(01)75884-5) (visited on 05/28/2025).
- [16] Juan C Caicedo et al. “Evaluation of deep learning strategies for nucleus segmentation in fluorescence images.” In: *Cytometry. Part A: the Journal of the International Society for Analytical Cytology* 95.9 (Sept. 2019), pp. 952–965. DOI: 10.1002/cyto.a.23863. URL: <http://dx.doi.org/10.1002/cyto.a.23863> (visited on 06/08/2025).
- [17] Shaoqing Ren et al. “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks.” In: *IEEE transactions on pattern analysis and machine intelligence* 39.6 (June 2017), pp. 1137–1149. DOI: 10.1109/TPAMI.2016.2577031. URL: <http://dx.doi.org/10.1109/TPAMI.2016.2577031> (visited on 05/08/2017).
- [18] Kaiming He et al. “Mask R-CNN”. In: *IEEE International Conference on Computer Vision (ICCV)*. IEEE, Oct. 2017, pp. 2980–2988. ISBN: 978-1-5386-1032-9. DOI: 10.1109/ICCV.2017.322. URL: <http://ieeexplore.ieee.org/document/8237584/> (visited on 03/03/2020).
- [19] Andreas Malthé Faber. *gpu-tracking*. 2023.
- [20] Daniel Sage et al. “Super-resolution fight club: assessment of 2D and 3D single-molecule localization microscopy software.” In: *Nature Methods* 16.5 (May 2019), pp. 387–395. ISSN: 1548-7091. DOI: 10.1038/s41592-019-0364-4. URL: <http://www.nature.com/articles/s41592-019-0364-4> (visited on 03/18/2025).
- [21] Antoine Yang et al. “TubeDETR: Spatio-Temporal Video Grounding with Transformers”. In: *arXiv* (2022). DOI: 10.48550/arxiv.2203.16434. URL: <https://arxiv.org/abs/2203.16434> (visited on 06/09/2025).
- [22] Aditya Khamparia and Karan Mehtab Singh. “A systematic review on deep learning architectures and applications”. In: *Expert Systems* 36.3 (June 2019), e12400. ISSN: 0266-4720. DOI: 10.1111/exsy.12400. URL: <https://onlinelibrary.wiley.com/doi/10.1111/exsy.12400> (visited on 06/09/2025).
- [23] Mohit Sewak, Sanjay K. Sahay, and Hemant Rathore. “An overview of deep learning architecture of deep neural networks and autoencoders”. In: *Journal of computational and theoretical nanoscience* 17.1 (Jan. 2020), pp. 182–188. ISSN: 1546-1955. DOI: 10.1166/jctn.2020.8648. URL: <https://www.ingentaconnect.com/content/10.1166/jctn.2020.8648> (visited on 06/09/2025).

- [24] Athanasios Voulodimos et al. “Deep learning for computer vision: A brief review.” In: *Computational intelligence and neuroscience* 2018 (Feb. 2018), p. 7068349. DOI: 10.1155/2018/7068349. URL: <http://dx.doi.org/10.1155/2018/7068349> (visited on 06/09/2025).
- [25] Elias Nehme et al. “Deep-STORM: super-resolution single-molecule microscopy by deep learning”. In: *Optica* 5.4 (Apr. 2018), p. 458. ISSN: 2334-2536. DOI: 10.1364/{OPTICA}.5.000458. URL: <https://www.osapublishing.org/abstract.cfm?%7BURI%7D=optica-5-4-458> (visited on 05/19/2025).
- [26] Ross Girshick et al. “R-CNN Rich feature hierarchies for accurate object detection and semantic segmentation”. In: *IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, June 2014, pp. 580–587. ISBN: 978-1-4799-5118-5. DOI: 10.1109/{CVPR}.2014.81. URL: <http://ieeexplore.ieee.org/document/6909475/> (visited on 06/07/2025).
- [27] *IEEE Xplore Full-Text PDF: WEBSITE*. URL: <https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=7410526> (visited on 06/07/2025).
- [28] Joseph Redmon et al. “YOLO You Only Look Once: Unified, Real-Time Object Detection”. In: *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, June 2016, pp. 779–788. ISBN: 978-1-4673-8851-1. DOI: 10.1109/{CVPR}.2016.91. URL: <http://ieeexplore.ieee.org/document/7780460/> (visited on 03/03/2020).
- [29] Oluwaseyi Olorunshola, Paul Jemitola, and Adeniran Ademuwagun. “Comparative Study of Some Deep Learning Object Detection Algorithms: R-CNN, FAST R-CNN, FASTER R-CNN, , and YOLO”. In: *Nile Journal of Engineering and Applied Science* 0 (2023), p. 1. ISSN: 3026-930X. DOI: 10.5455/{NJEAS}.150264. URL: <https://ejmanager.com/fulltextpdf.php?mno=150264> (visited on 06/09/2025).
- [30] Bastian Th Eichenberger et al. “deepBlink: threshold-independent detection and localization of diffraction-limited spots.” In: *Nucleic Acids Research* 49.13 (July 2021), pp. 7292–7297. DOI: 10.1093/nar/gkab546. URL: <http://dx.doi.org/10.1093/nar/gkab546> (visited on 05/24/2025).
- [31] I F Sbalzarini and P Koumoutsakos. “Feature point tracking and trajectory analysis for video imaging in cell biology.” In: *Journal of Structural Biology* 151.2 (Aug. 2005), pp. 182–195. DOI: 10.1016/j.jsb.2005.06.002. URL: <http://dx.doi.org/10.1016/j.jsb.2005.06.002> (visited on 05/28/2025).
- [32] C. M. Brown. “Fluorescence microscopy avoiding the pitfalls”. In: *Journal of cell science* 120.19 (Oct. 2007), pp. 3488–3488. ISSN: 0021-9533. DOI: 10.1242/jcs.022079. URL: <http://jcs.biologists.org/cgi/doi/10.1242/jcs.022079> (visited on 06/09/2025).
- [33] Jennifer C Waters. “Accuracy and precision in quantitative fluorescence microscopy.” In: *The Journal of Cell Biology* 185.7 (June 2009), pp. 1135–1148. DOI: 10.1083/jcb.200903097. URL: <http://dx.doi.org/10.1083/jcb.200903097> (visited on 03/16/2023).
- [34] Magdalena C. Schneider et al. “Interactive simulation and visualization of point spread functions in single molecule imaging”. In: *Optics communications* 560 (June 2024), p. 130463. ISSN: 00304018. DOI: 10.1016/j.optcom.2024.130463. URL: <https://linkinghub.elsevier.com/retrieve/pii/S0030401824002001> (visited on 03/18/2025).
- [35] Sarah Frisken Gibson and Frederick Lanni. “Experimental test of an analytical model of aberration in an oil-immersion objective lens used in three-dimensional light microscopy”. In: *J. Opt. Soc. Am. A* 9.1 (Jan. 1992), pp. 154–166. DOI: 10.1364/JOSAA.9.000154. URL: <https://opg.optica.org/josaa/abstract.cfm?URI=josaa-9-1-154>.

- [36] B. Richards and E. Wolf. “Electromagnetic Diffraction in Optical Systems. II. Structure of the Image Field in an Aplanatic System”. In: *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences* 253.1274 (Dec. 1959), pp. 358–379. ISSN: 1364-5021. DOI: 10.1098/rspa.1959.0200. URL: <http://rspa.royalsocietypublishing.org/cgi/doi/10.1098/rspa.1959.0200> (visited on 06/07/2025).
- [37] *Hamamatsu Learning Center: CCD Noise Sources and Signal-to-Noise Ratio*. WEBSITE. URL: https://hamamatsu.magnet.fsu.edu/articles/ccdsnr.html?utm%5C_source=chatgpt.com (visited on 06/09/2025).
- [38] *Signal-to-Noise Considerations*. WEBSITE. URL: <https://evidentscientific.com/en/microscope-resource/knowledge-hub/techniques/confocal/signaltonoise> (visited on 06/09/2025).

6 Appendix

All code is available on GitHub: <https://github.com/890tsugua/bachelor1>

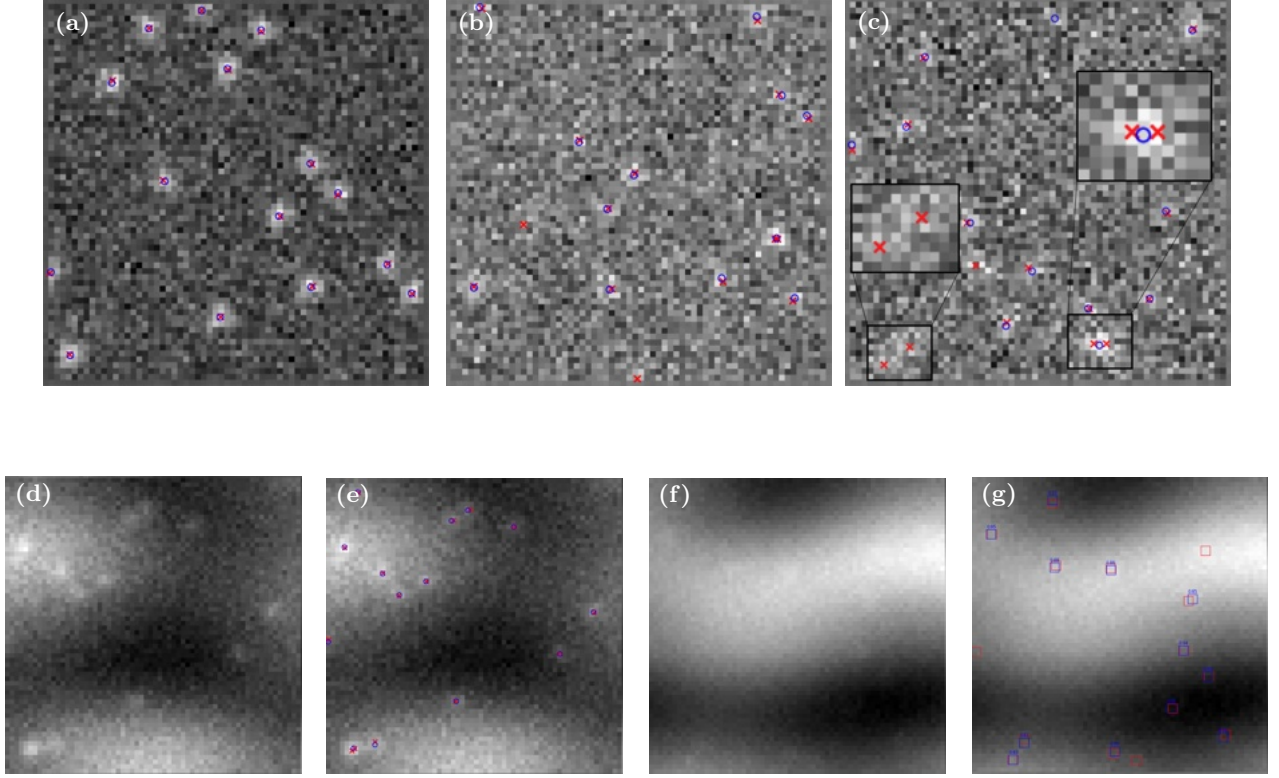


Figure 8: Model predictions on homogeneous (a-c) and heterogeneous (d-g) backgrounds. Ground truths are red and predictions are blue. **a)** The model works well at low density and SNR around 5-6. **b)** At SNR around 3, the model is still able to detect most emitters. **c)** At low SNR, around 2-3, false negatives are more common. In this image, the model struggles to recognize emitters grouped closely together. **d-e)** The model performs well on heterogeneous noise at high SNR. **f-g)** At very high background intensities and heterogeneous noise, it is practically impossible for the naked eye to make out emitters due to scaling of the image, even though the SNR is medium (here around 4-6). The spots could be made visible by finetuning brightness and contrast, but this is quite tedious for heterogeneous noise. The model however is able to detect the spots with high accuracy.

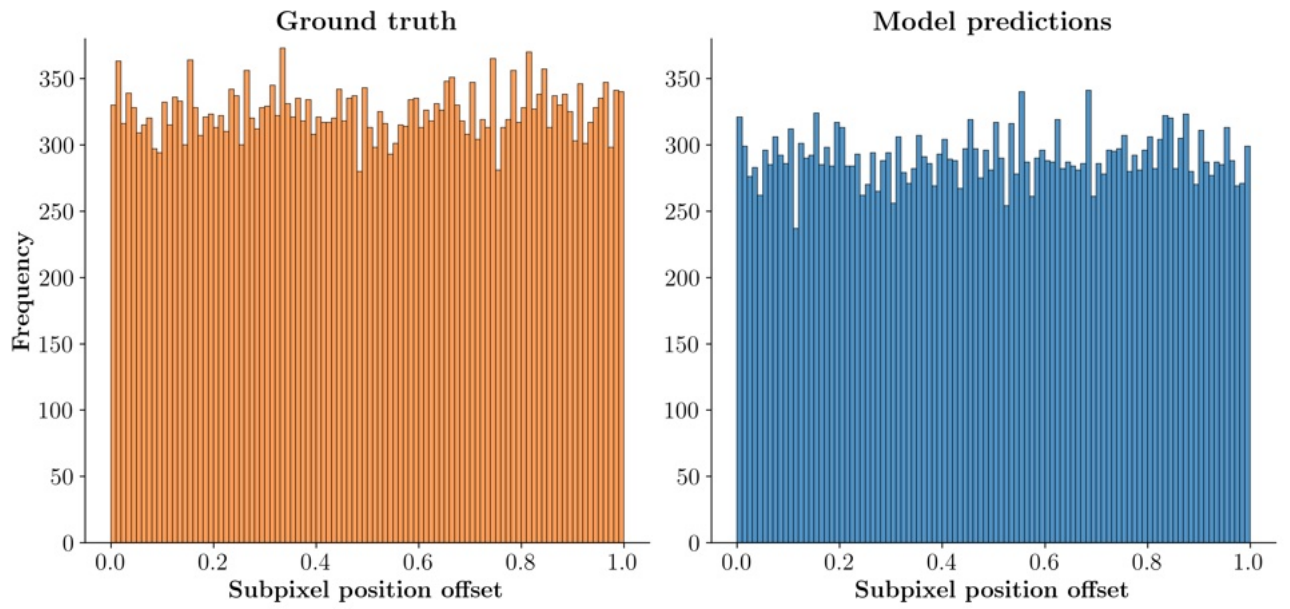


Figure 9: Distribution of the sub-pixel offset values. The random, uniform distribution shows that the model does not have a bias for making predictions in the middle of pixels, which would be an issue.

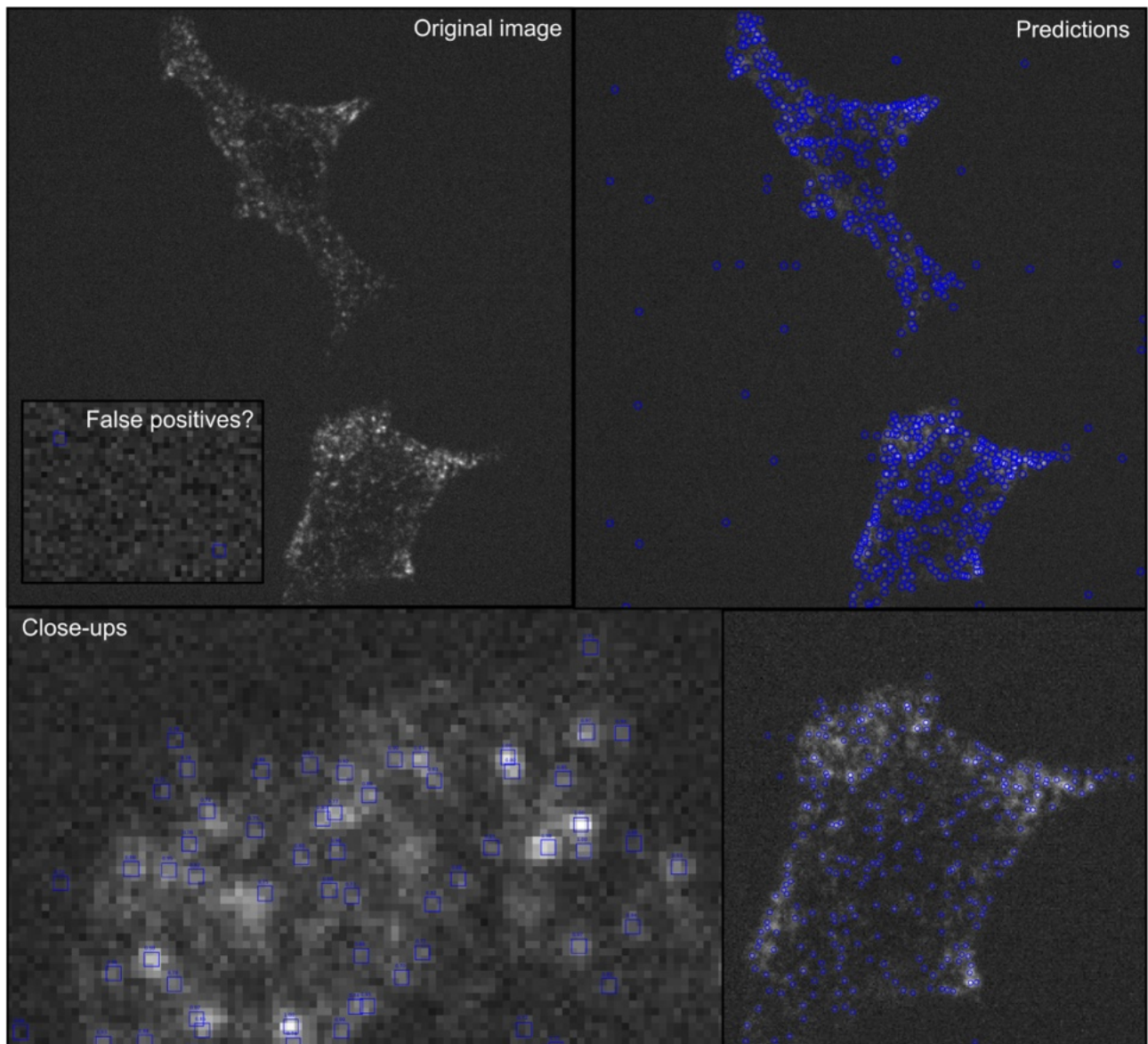


Figure 10: Model localization of transferrin proteins in HeLa cells. Image courtesy of Sara V. Bleshø

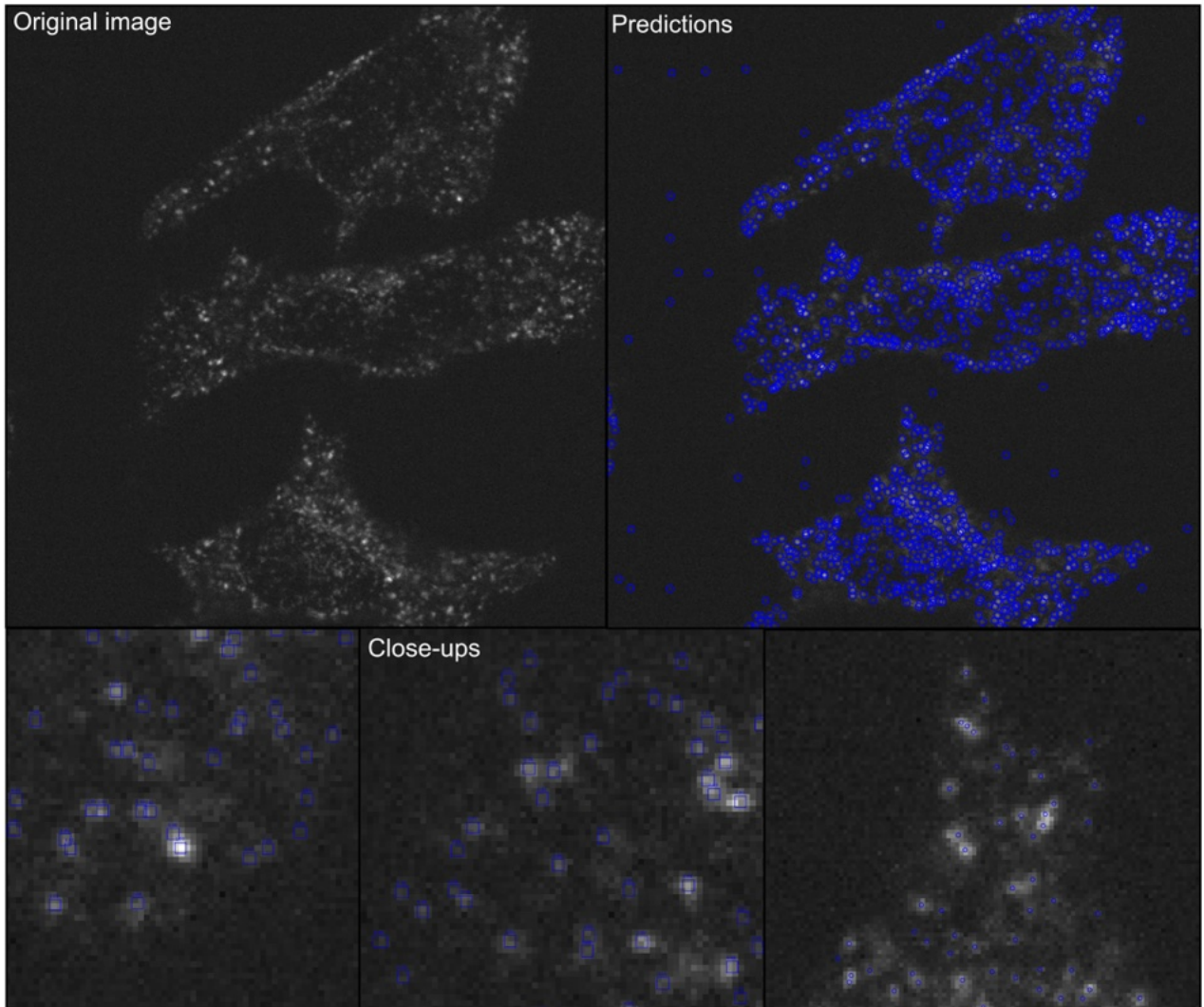


Figure 11: Model localization of transferrin proteins in HeLa cells. Image courtesy of Sara V. Bleshøy

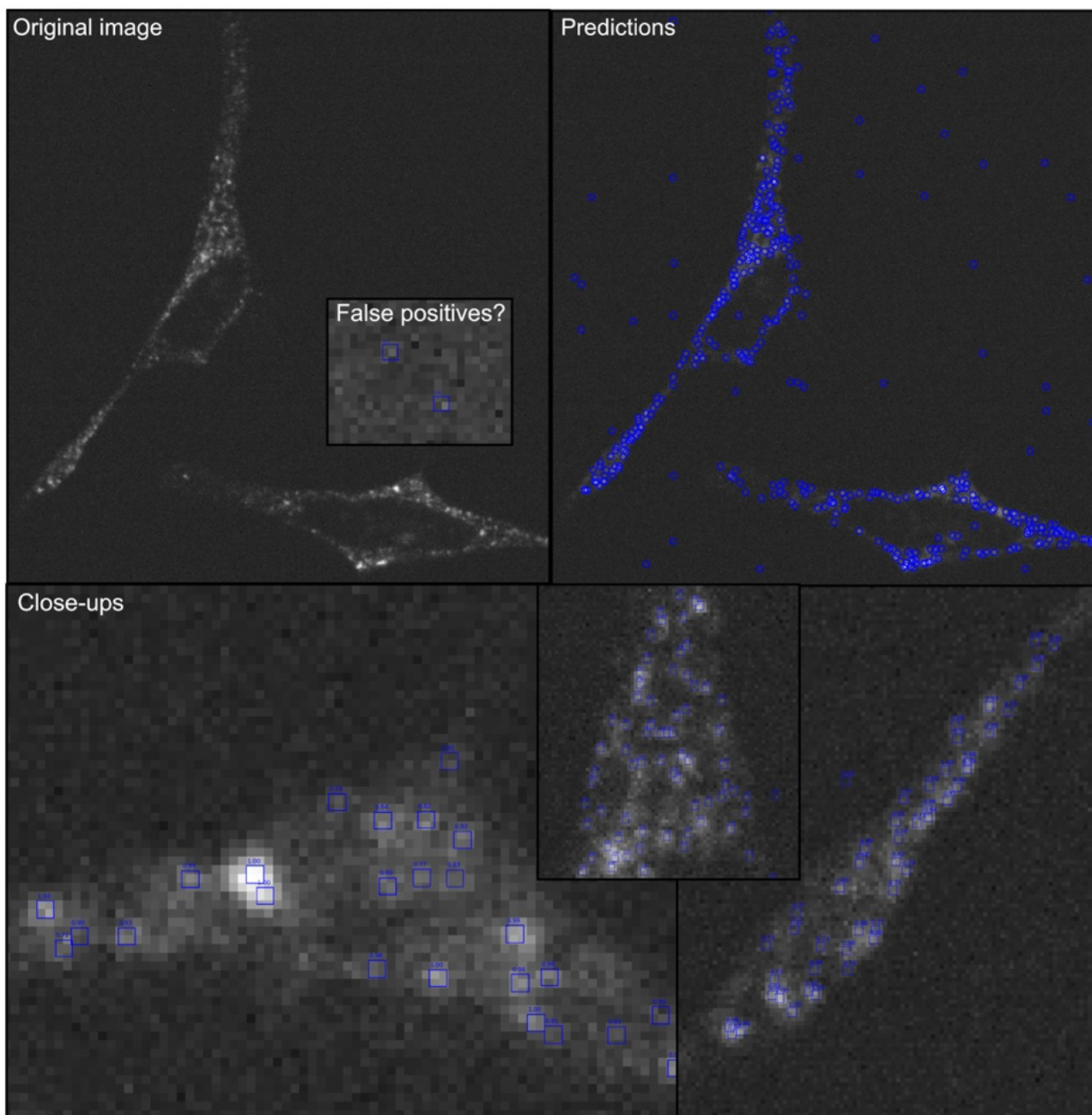


Figure 12: Model localization of transferrin proteins in HeLa cells. Image courtesy of Sara V. Bleshøy

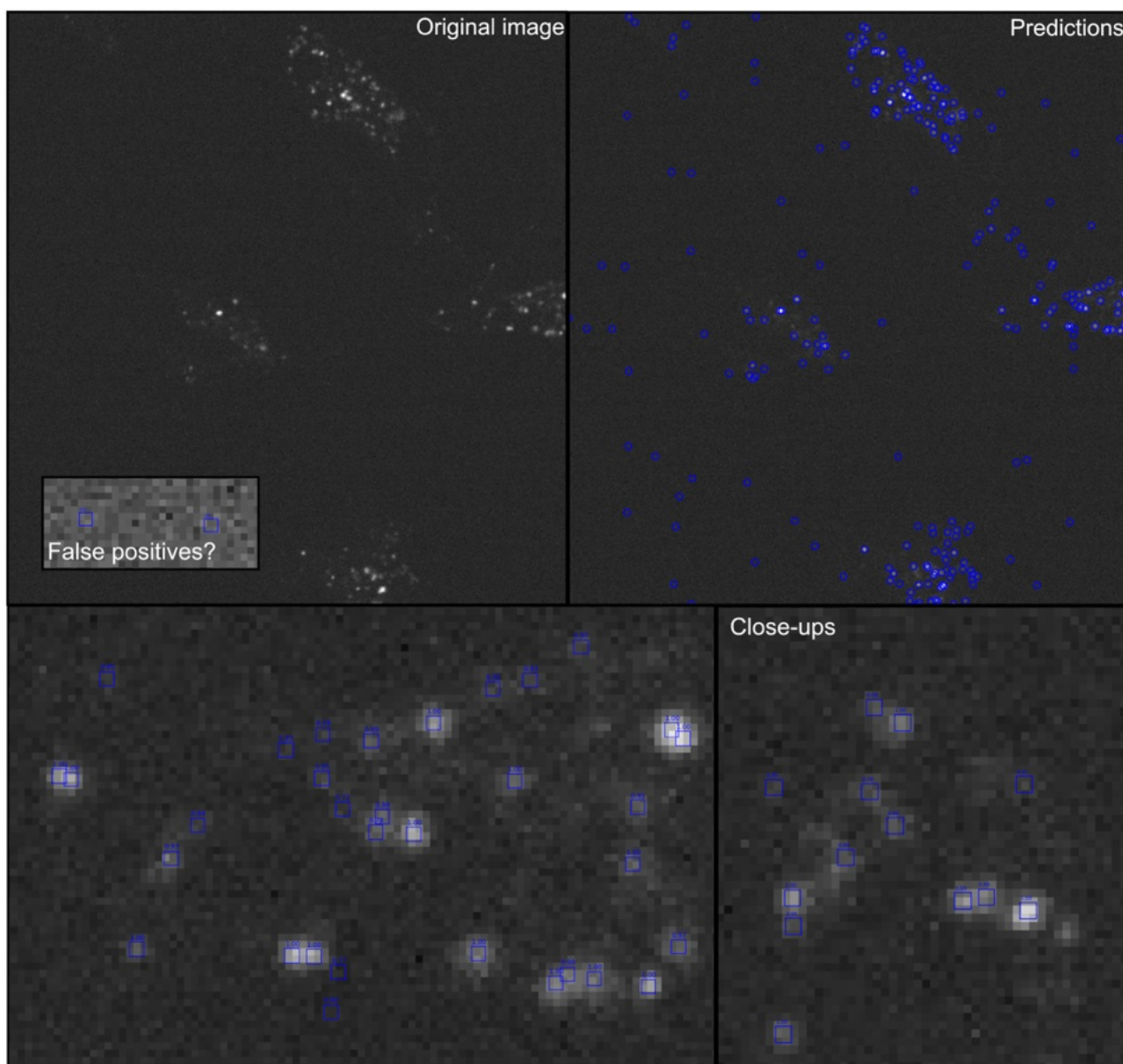


Figure 13: Model localization of lysosomes in HeLa cells. Image courtesy of Sara V. Bleshøy

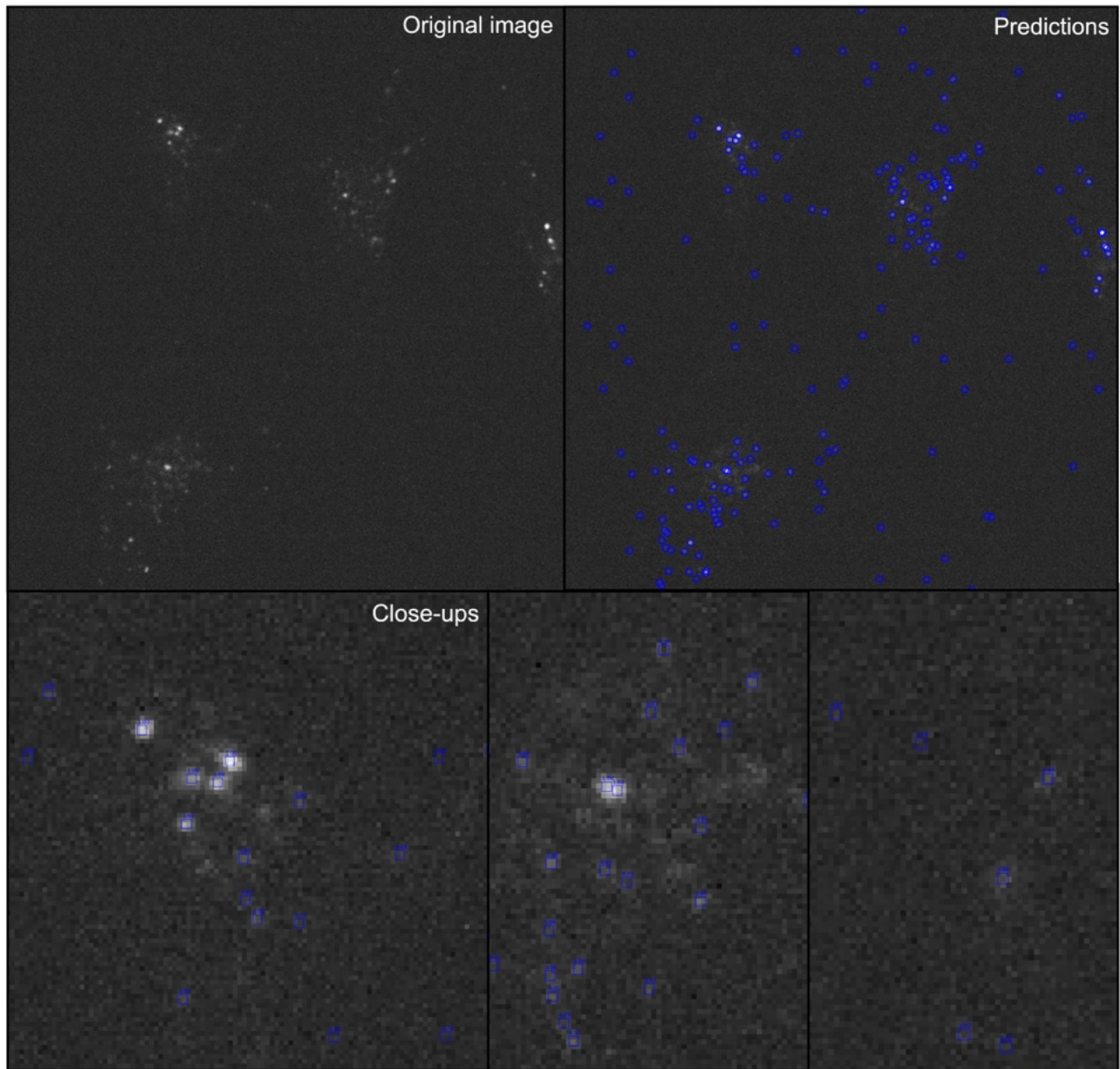


Figure 14: Model localization of lysosomes in HeLa cells. Image courtesy of Sara V. Bleshøy

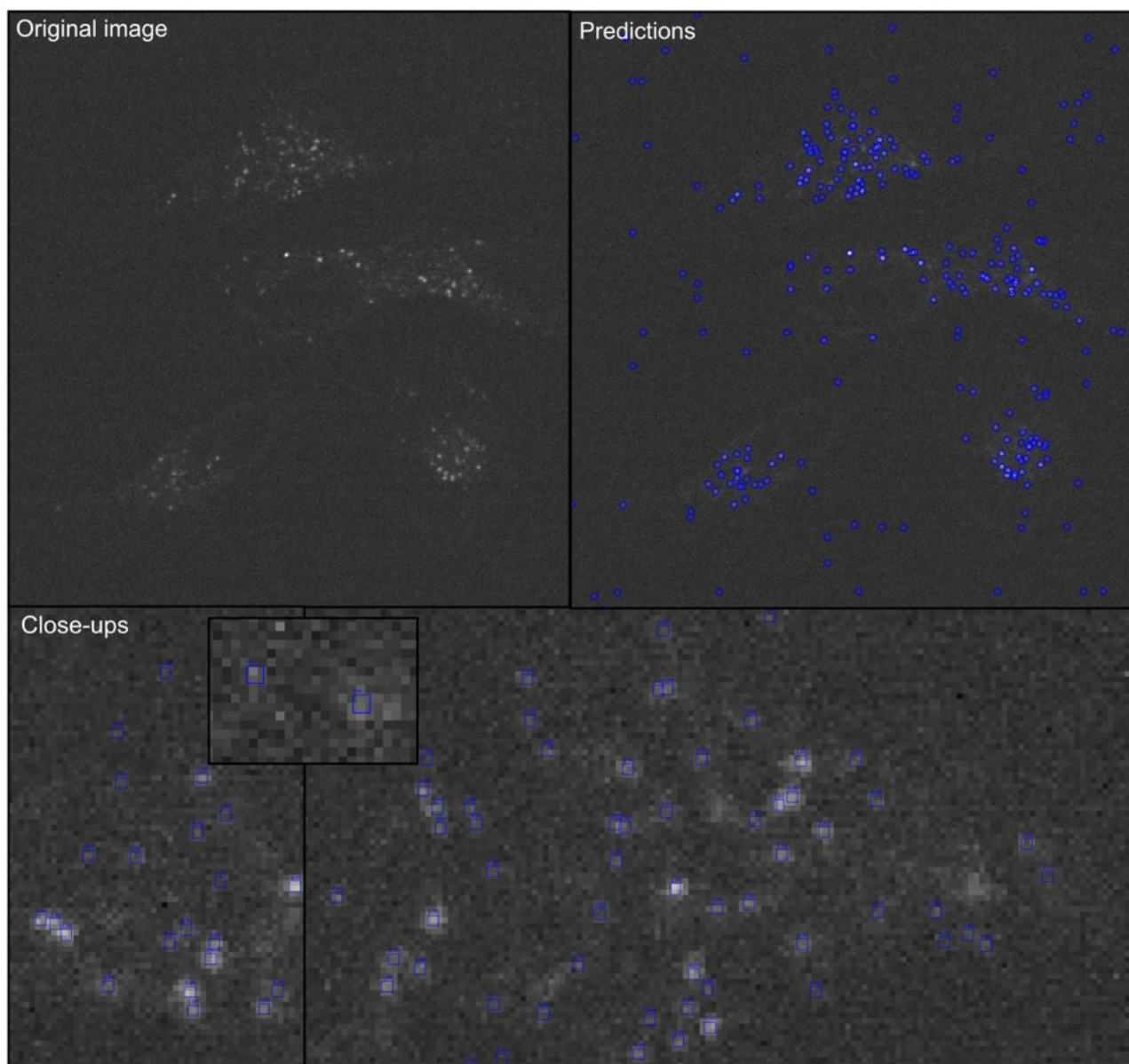


Figure 15: Model localization of lysosomes in HeLa cells. Image courtesy of Sara V. Bleshøy

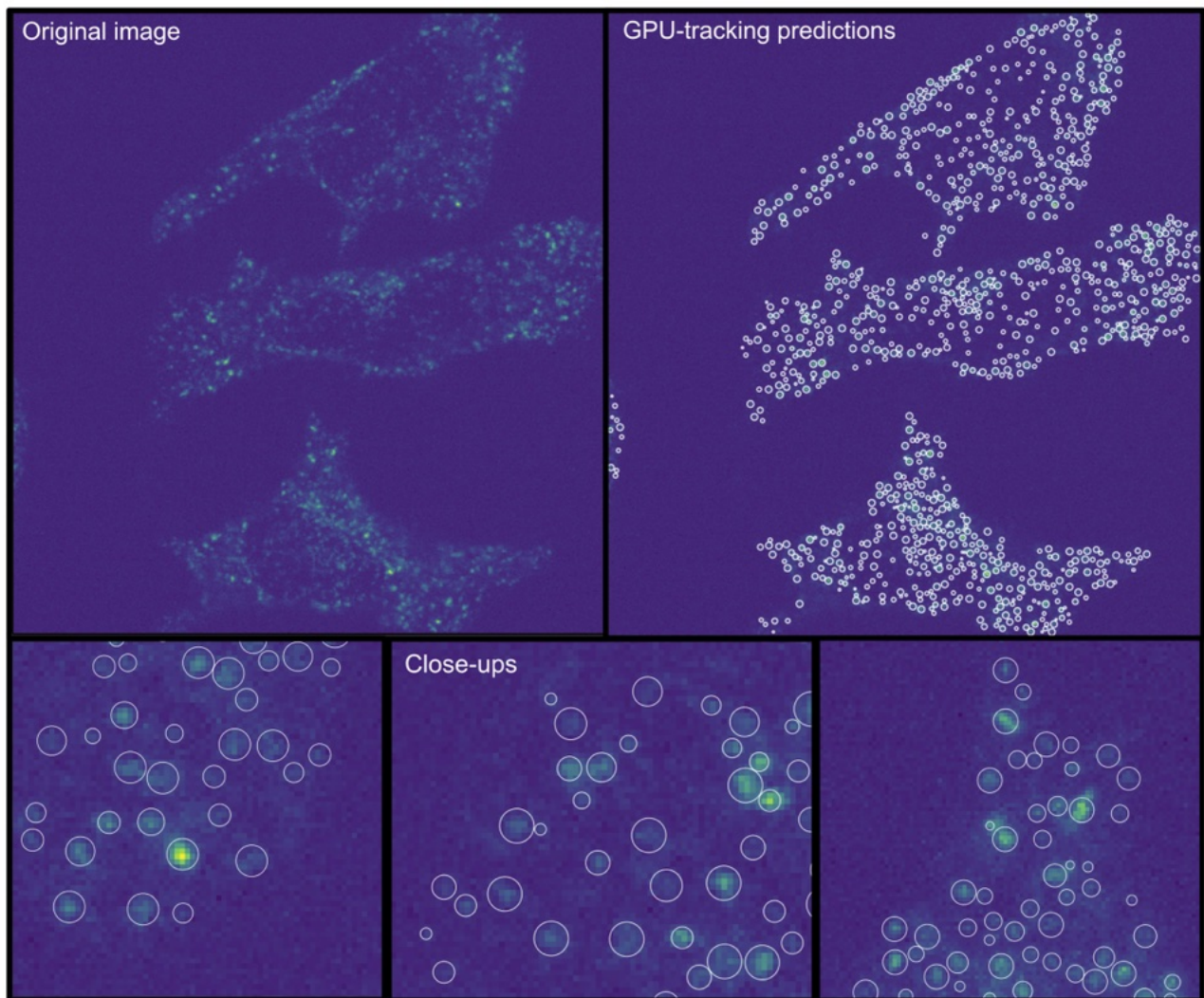


Figure 16: Gpu-tracking localization of transferrin proteins in HeLa cells (same as in Fig. 6. Image courtesy of Sara V. Bleshøy

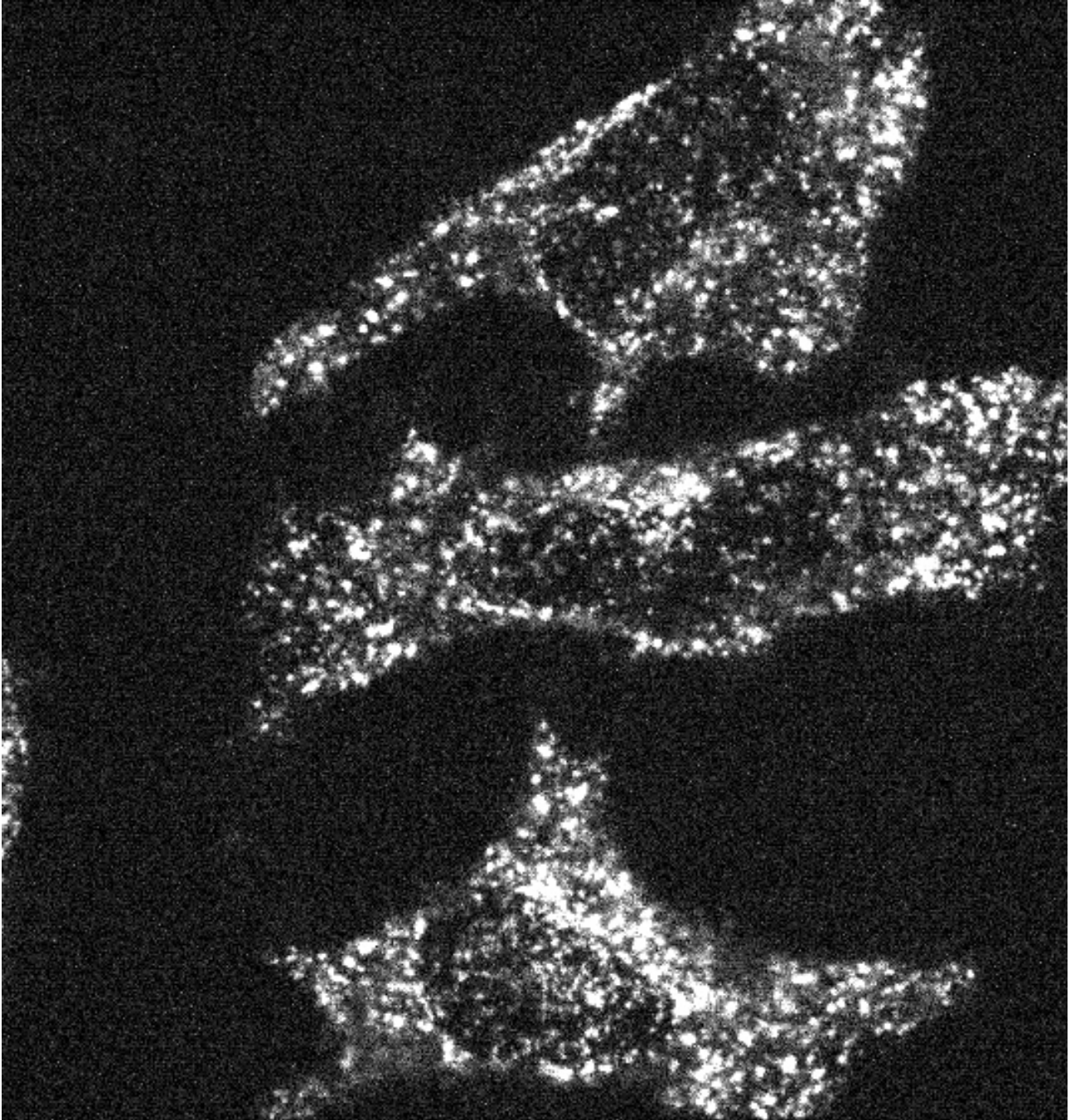


Figure 17: Brightness/contrast enhanced image of Figure 6. Image courtesy of Sara V. Bleshøy

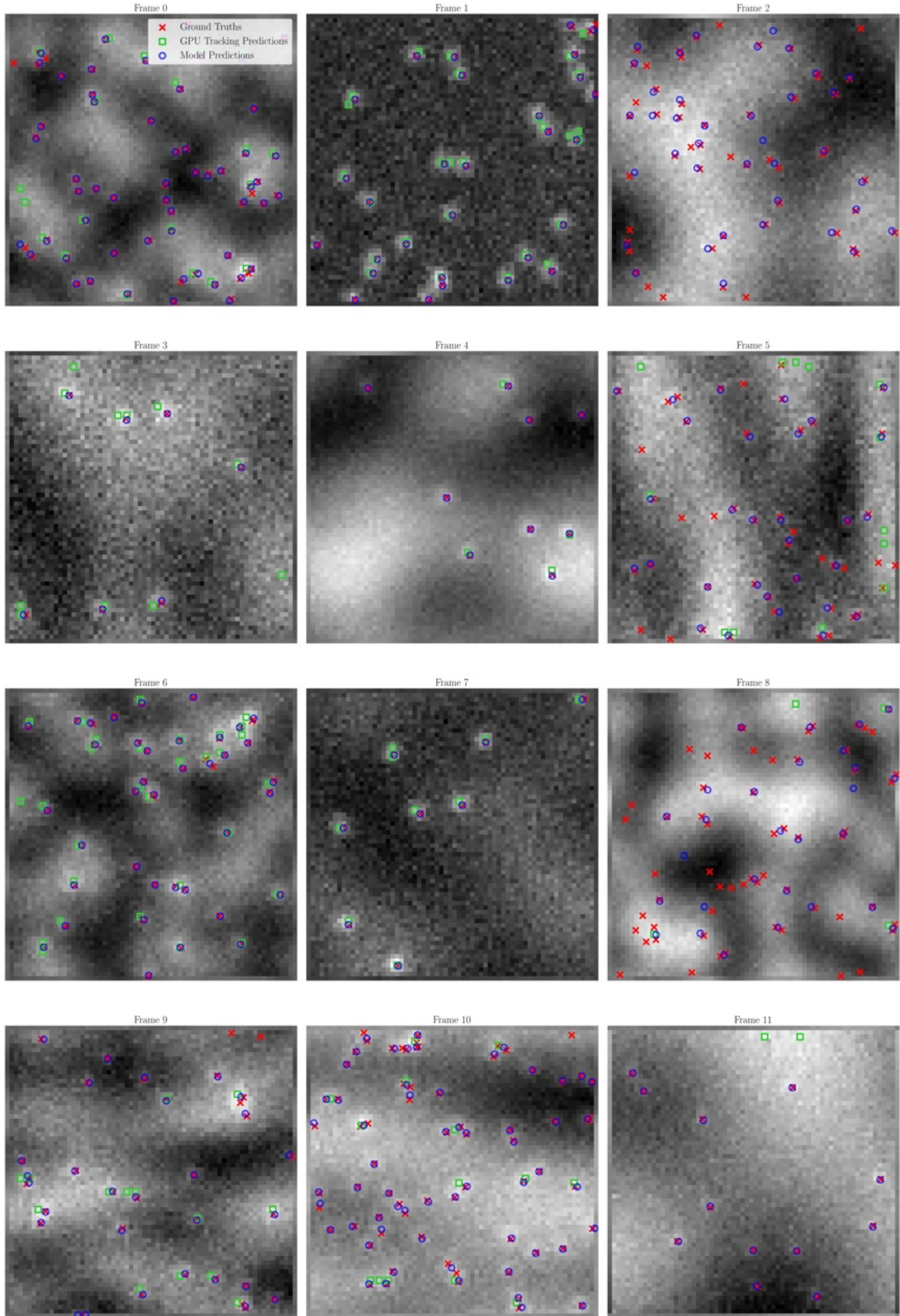


Figure 18: Examples of model and gpu-tracking predictions on simulated data. Ground truths are read, model predictions are blue and gpu-tracking predictions are green. Gpu-tracking struggles to make predictions in highly heterogeneous noise environments.

Deklaration for anvendelse af generative AI-værktøjer (studerende)

☒ Jeg/vi har benyttet generativ AI som hjælpemiddel/værktøj (sæt kryds)

☐ Jeg/vi har **IKKE** benyttet generativ AI som hjælpemiddel/værktøj (sæt kryds)

Hvis brug af generativ AI er tilladt til eksamen, men du ikke har benyttet det i din opgave, skal du blot krydse af, at du ikke har brugt GAI, og behøver ikke at udfylde resten.

Oplist, hvilke GAI-værktøjer der er benyttet, inkl. link til platformen (hvis muligt):

ChatGPT (OpenAI): www.chatgpt.com

GitHub CoPilot: <https://github.com/features/copilot>

Google AI Overviews

Beskriv hvordan generativ AI er anvendt i opgaven:

1) Purpose: ChatGPT and copilot were used to generate code for simple functions, eg. Plotting or array manipulation. It was also used for debugging and for recommendations on some more complicated parts of the code. ChatGPT and AI overviews have also been used as general search tools.

2) Work phase: GAI has been used in preliminary research and for programming.

3) Most of the time, code generated with CoPilot or ChatGPT was further modified, either to fix issues with the code or to change functionality.

NB. GAI-genereret indhold brugt som kilde i opgaven kræver korrekt brug af citationstegn og kildehenvisning. Læs retningslinjer fra Københavns Universitetsbibliotek på KUnet.